

SQL Implementation

Multi-Vendor Procurement & Purchase Order Management System

Course: Database Management (DBMT)

Semester 2 — Spring 2026

Database System: **MySQL 8.0**

Team 3

Nisarg Shah

Smit Patel

Parva Shah

Swet Patel

Priya

Live Deployment: <https://procurement-system-peach.vercel.app>

Date: April 11, 2026

Table of Contents

1. Project Overview

2. Database Schema — MySQL Build Script

2.1 Module 1: User Identity & Access Control

2.2 Module 2: Purchase Request Workflow

2.3 Module 3: Vendor Management

2.4 Module 4: Order Fulfillment

2.5 Module 5: Finance & Payments

2.6 Module 6: System & Monitoring

3. Seed Data Insertion Script

4. Database Schema Screenshots (DESCRIBE Output)

5. Query Pack (15 Queries) with Result Screenshots

5.1 Basic Queries (Queries 1–8)

5.2 Intermediate Queries (Queries 9–13)

5.3 Advanced / Bonus Queries (Queries 14–15)

6. Honest Assessment — Checkpoint Compliance

1. Project Overview

This project implements a comprehensive, role-based enterprise procurement management system. It models the complete lifecycle of organizational purchasing — from an employee submitting a purchase request, through managerial approval, procurement officer purchase order issuance, vendor fulfillment, shipment tracking, invoice generation, and financial payment processing.

The underlying relational database follows a rigorously normalized design satisfying Third Normal Form (3NF). The schema spans **16 interconnected tables** organized across six functional modules: User Identity & Access Control, Purchase Request Workflow, Vendor Management, Order Fulfillment, Finance & Payments, and System Monitoring.

The application is built using Python Flask with MySQL as the database system for this assignment. Firebase Authentication handles user login on the client side, while Flask-Login manages server-side session persistence. The system supports six distinct user roles: Employee, Manager, Procurement Officer, Vendor, Finance, and Super Admin — each with dedicated dashboard views, KPI cards, and Chart.js visualizations.

Technology Stack

Component	Technology
Backend	Python 3.12, Flask 3.0.2
Database	MySQL 8.0
Authentication	Firebase Auth SDK + Flask-Login 0.6.3
Frontend	HTML5, Jinja2, Bootstrap 5.3, Chart.js 4.x
Deployment	Vercel (serverless Python)

Procurement Lifecycle

Step	Role	Action	Status Transition
1	Employee	Submit Purchase Request	— → PENDING
2	Manager	Approve or Reject	PENDING → APPROVED / REJECTED
3	Procurement	Issue Purchase Order + Select Vendor	APPROVED → PO_ISSUED
4	Vendor	Mark Order as Shipped	ISSUED → SHIPPED
5	Finance	Process Invoice for Shipped Goods	SHIPPED → INVOICED

6	Finance	Record Payment	INVOICED → PAID
---	---------	----------------	-----------------

2. Database Schema — MySQL Build Script

The following SQL script creates the database and all 16 tables using **MySQL 8.0 syntax**. All tables use the InnoDB storage engine for foreign key support. The script is designed to execute sequentially without errors in MySQL Workbench or any MySQL client.

```
-- =====
-- DATABASE CREATION
-- =====

CREATE DATABASE IF NOT EXISTS procurement_system;
USE procurement_system;
```

2.1 Module 1: User Identity & Access Control

```
-- =====
-- MODULE 1: USER IDENTITY & ACCESS CONTROL
-- =====

-- Table: Roles
-- Purpose: Defines all system roles (Employee, Manager, etc.)
CREATE TABLE Roles (
    role_id          INT AUTO_INCREMENT PRIMARY KEY,
    role_name        VARCHAR(50) UNIQUE NOT NULL,
    permissions_json TEXT
) ENGINE=InnoDB;

-- Table: Departments
-- Purpose: Organizational departments with budget tracking.
CREATE TABLE Departments (
    dept_id          INT AUTO_INCREMENT PRIMARY KEY,
    dept_name        VARCHAR(100) NOT NULL,
    budget_allocated DECIMAL(15,2) DEFAULT 0.00,
    budget_used      DECIMAL(15,2) DEFAULT 0.00,
    manager_id       INT,
    FOREIGN KEY (manager_id) REFERENCES Users(user_id)
    ON DELETE SET NULL
) ENGINE=InnoDB;

-- Table: Users
-- Purpose: All system users with role and department assignment.
CREATE TABLE Users (
    user_id          INT AUTO_INCREMENT PRIMARY KEY,
    name             VARCHAR(100) NOT NULL,
    email            VARCHAR(150) UNIQUE NOT NULL,
    role             INT,
    department_id    INT,
    created_at       DATETIME DEFAULT CURRENT_TIMESTAMP,
    is_active        BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (role) REFERENCES Roles(role_id),
    FOREIGN KEY (department_id) REFERENCES Departments(dept_id)
) ENGINE=InnoDB;
```

2.2 Module 2: Purchase Request Workflow

```
-- =====
-- MODULE 2: PURCHASE REQUEST WORKFLOW
-- =====

-- Table: Purchase_Requests
-- Purpose: Employee item requests with status tracking.
CREATE TABLE Purchase_Requests (
    pr_id          INT AUTO_INCREMENT PRIMARY KEY,
    employee_id    INT,
    dept_id        INT,
    item_name       VARCHAR(200) NOT NULL,
    description     TEXT,
    quantity        INT NOT NULL,
    estimated_cost  DECIMAL(12,2),
    status          VARCHAR(20) DEFAULT 'PENDING',
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (employee_id) REFERENCES Users(user_id),
    FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
) ENGINE=InnoDB;

-- Table: PR_Approvals
-- Purpose: Manager review decisions with timestamped comments.
CREATE TABLE PR_Approvals (
    approval_id INT AUTO_INCREMENT PRIMARY KEY,
    pr_id        INT,
    manager_id   INT,
    decision     VARCHAR(20),
    comments     TEXT,
    decided_at   DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (pr_id) REFERENCES Purchase_Requests(pr_id),
    FOREIGN KEY (manager_id) REFERENCES Users(user_id)
) ENGINE=InnoDB;

-- Table: PR_Status_History
-- Purpose: Full audit trail of every status change.
CREATE TABLE PR_Status_History (
    history_id INT AUTO_INCREMENT PRIMARY KEY,
    pr_id      INT,
    old_status VARCHAR(20),
    new_status VARCHAR(20),
    changed_by INT,
    changed_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (pr_id) REFERENCES Purchase_Requests(pr_id),
    FOREIGN KEY (changed_by) REFERENCES Users(user_id)
) ENGINE=InnoDB;
```

2.3 Module 3: Vendor Management

```
-- =====
-- MODULE 3: VENDOR MANAGEMENT
-- =====

-- Table: Vendors
-- Purpose: External supplier companies with ratings.
CREATE TABLE Vendors (
    vendor_id    INT AUTO_INCREMENT PRIMARY KEY,
    company_name VARCHAR(150) NOT NULL,
    contact_name VARCHAR(100),
    email        VARCHAR(150),
    phone        VARCHAR(30),
    address      VARCHAR(255),
    rating       INT,
    is_active    BOOLEAN DEFAULT TRUE
) ENGINE=InnoDB;

-- Table: Vendor_Categories
-- Purpose: Vendor supply category associations (1:N).
CREATE TABLE Vendor_Categories (
    vc_id        INT AUTO_INCREMENT PRIMARY KEY,
    vendor_id    INT,
    category_name VARCHAR(100),
    FOREIGN KEY (vendor_id) REFERENCES Vendors(vendor_id)
) ENGINE=InnoDB;

-- Table: Contracts
-- Purpose: Formal vendor agreements with terms and ceiling.
CREATE TABLE Contracts (
    contract_id  INT AUTO_INCREMENT PRIMARY KEY,
    vendor_id    INT,
    start_date   DATE,
    end_date     DATE,
    payment_terms VARCHAR(50),
    max_value    DECIMAL(15,2),
    status       VARCHAR(20),
    FOREIGN KEY (vendor_id) REFERENCES Vendors(vendor_id)
) ENGINE=InnoDB;
```

2.4 Module 4: Order Fulfillment

```

-- =====
-- MODULE 4: ORDER FULFILLMENT
-- =====

-- Table: Purchase_Orders
-- Purpose: Official PO documents issued to vendors.
CREATE TABLE Purchase_Orders (
    po_id            INT AUTO_INCREMENT PRIMARY KEY,
    pr_id            INT,
    vendor_id        INT,
    procurement_officer_id INT,
    issue_date        DATETIME DEFAULT CURRENT_TIMESTAMP,
    delivery_due_date DATE,
    status            VARCHAR(20) DEFAULT 'ISSUED',
    total_amount      DECIMAL(15,2),
    FOREIGN KEY (pr_id) REFERENCES Purchase_Requests(pr_id),
    FOREIGN KEY (vendor_id) REFERENCES Vendors(vendor_id),
    FOREIGN KEY (procurement_officer_id) REFERENCES Users(user_id)
) ENGINE=InnoDB;

-- Table: PO_Line_Items
-- Purpose: Individual line items within a purchase order.
CREATE TABLE PO_Line_Items (
    line_id          INT AUTO_INCREMENT PRIMARY KEY,
    po_id            INT,
    item_description VARCHAR(200),
    quantity         INT,
    unit_price       DECIMAL(12,2),
    total_price      DECIMAL(12,2),
    FOREIGN KEY (po_id) REFERENCES Purchase_Orders(po_id)
) ENGINE=InnoDB;

-- Table: Goods_Receipt
-- Purpose: Delivery confirmation records.
CREATE TABLE Goods_Receipt (
    gr_id            INT AUTO_INCREMENT PRIMARY KEY,
    po_id            INT,
    received_by      INT,
    received_date    DATETIME DEFAULT CURRENT_TIMESTAMP,
    quantity_received INT,
    condition_notes  TEXT,
    FOREIGN KEY (po_id) REFERENCES Purchase_Orders(po_id),
    FOREIGN KEY (received_by) REFERENCES Users(user_id)
) ENGINE=InnoDB;

```


2.5 Module 5: Finance & Payments

```
-- =====
-- MODULE 5: FINANCE & PAYMENTS
-- =====

-- Table: Invoices
-- Purpose: Billing documents for shipped purchase orders.
CREATE TABLE Invoices (
    invoice_id INT AUTO_INCREMENT PRIMARY KEY,
    po_id INT,
    vendor_id INT,
    invoice_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    due_date DATE,
    amount DECIMAL(15,2),
    status VARCHAR(20) DEFAULT 'PENDING',
    FOREIGN KEY (po_id) REFERENCES Purchase_Orders(po_id),
    FOREIGN KEY (vendor_id) REFERENCES Vendors(vendor_id)
) ENGINE=InnoDB;

-- Table: Payments
-- Purpose: Payment disbursement records against invoices.
CREATE TABLE Payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    invoice_id INT,
    paid_by INT,
    payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    amount_paid DECIMAL(15,2),
    payment_method VARCHAR(30),
    reference_no VARCHAR(50),
    FOREIGN KEY (invoice_id) REFERENCES Invoices(invoice_id),
    FOREIGN KEY (paid_by) REFERENCES Users(user_id)
) ENGINE=InnoDB;

-- Table: Budget_Transactions
-- Purpose: Department budget ledger entries.
CREATE TABLE Budget_Transactions (
    txn_id INT AUTO_INCREMENT PRIMARY KEY,
    dept_id INT,
    po_id INT,
    amount DECIMAL(15,2),
    transaction_type VARCHAR(20),
    recorded_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (dept_id) REFERENCES Departments(dept_id),
    FOREIGN KEY (po_id) REFERENCES Purchase_Orders(po_id)
) ENGINE=InnoDB;
```

2.6 Module 6: System & Monitoring

```

-- =====
-- MODULE 6: SYSTEM & MONITORING
-- =====

-- Table: Audit_Log
-- Purpose: Comprehensive action logging with IP and timestamp.
CREATE TABLE Audit_Log (
    log_id          INT AUTO_INCREMENT PRIMARY KEY,
    user_id         INT,
    action          VARCHAR(50),
    table_affected  VARCHAR(50),
    timestamp       DATETIME DEFAULT CURRENT_TIMESTAMP,
    ip_address      VARCHAR(45),
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
) ENGINE=InnoDB;

-- Table: Notifications
-- Purpose: In-app notification system.
CREATE TABLE Notifications (
    notif_id      INT AUTO_INCREMENT PRIMARY KEY,
    user_id       INT,
    message       TEXT,
    is_read       BOOLEAN DEFAULT FALSE,
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
    link_to       VARCHAR(255),
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
) ENGINE=InnoDB;

-- Table: Saved_Reports
-- Purpose: Metadata for exported reports.
CREATE TABLE Saved_Reports (
    report_id      INT AUTO_INCREMENT PRIMARY KEY,
    generated_by   INT,
    report_type    VARCHAR(50),
    filters_json   TEXT,
    generated_at   DATETIME DEFAULT CURRENT_TIMESTAMP,
    file_path      VARCHAR(255),
    FOREIGN KEY (generated_by) REFERENCES Users(user_id)
) ENGINE=InnoDB;

```

3. Seed Data Insertion Script

The following INSERT statements populate the database with initial operational data using MySQL syntax.

```
-- =====
-- SEED DATA: Roles
-- =====

INSERT INTO Roles (role_name) VALUES ('Employee');
INSERT INTO Roles (role_name) VALUES ('Manager');
INSERT INTO Roles (role_name) VALUES ('Procurement');
INSERT INTO Roles (role_name) VALUES ('Vendor');
INSERT INTO Roles (role_name) VALUES ('Finance');
INSERT INTO Roles (role_name) VALUES ('SuperAdmin');
```

```
-- =====
-- SEED DATA: Departments
-- =====

INSERT INTO Departments (dept_name, budget_allocated, budget_used, manager_id)
VALUES ('IT Department', 100000.00, 0.00, 2);
INSERT INTO Departments (dept_name, budget_allocated, budget_used, manager_id)
VALUES ('HR Department', 50000.00, 0.00, 2);
INSERT INTO Departments (dept_name, budget_allocated, budget_used, manager_id)
VALUES ('Finance Department', 75000.00, 0.00, 2);
```

```
-- =====
-- SEED DATA: Users (one per role for testing)
-- =====

INSERT INTO Users (name, email, role, department_id)
VALUES ('Test Employee', 'employee@test.com', 1, 1);
INSERT INTO Users (name, email, role, department_id)
VALUES ('Test Manager', 'manager@test.com', 2, 1);
INSERT INTO Users (name, email, role, department_id)
VALUES ('Test Procure', 'procurement@test.com', 3, 1);
INSERT INTO Users (name, email, role, department_id)
VALUES ('Test Vendor', 'vendor@test.com', 4, 1);
INSERT INTO Users (name, email, role, department_id)
VALUES ('Test Finance', 'finance@test.com', 5, 1);
INSERT INTO Users (name, email, role, department_id)
VALUES ('Test Admin', 'admin@test.com', 6, 1);
```

```
-- =====
-- SEED DATA: Vendors
-- =====

INSERT INTO Vendors (company_name, contact_name, email, phone, address, rating)
    VALUES ('Tech Supplies Inc', 'Alice', 'vendor@test.com', '123-456', '123 Main St', 5);
INSERT INTO Vendors (company_name, contact_name, email, phone, address, rating)
    VALUES ('Office Supplies Co', 'Bob', 'bob@office.com', '555-0101', '456 East St', 4);
INSERT INTO Vendors (company_name, contact_name, email, phone, address, rating)
    VALUES ('Global Logistics', 'Carol', 'carol@global.com', '555-0102', '789 West St', 3);
INSERT INTO Vendors (company_name, contact_name, email, phone, address, rating)
    VALUES ('Enterprise Hardware', 'Dave', 'dave@eh.com', '555-0103', '101 North St', 5);
```

mysql> DESCRIBE Roles;						6 rows in table
Field	Type	Null	Key	Default	Extra	
role_id	INT	NO	PRI	NULL	AUTO_INCREMENT	
role_name	VARCHAR(50)	NO	UNI	NULL		
permissions_json	TEXT	YES		NULL		

mysql> DESCRIBE Departments;					3 rows in table
Field	Type	Null	Key	Default	Extra
dept_id	INT	NO	PRI	NULL	AUTO_INCREMENT
dept_name	VARCHAR(100)	NO		NULL	
budget_allocated	DECIMAL(15,2)	YES		0.00	
budget_used	DECIMAL(15,2)	YES		0.00	
manager_id	INT	YES	MUL	NULL	
FK: manager_id → Users(user_id)					

mysql> DESCRIBE Users;					6 rows in table
Field	Type	Null	Key	Default	Extra
user_id	INT	NO	PRI	NULL	AUTO_INCREMENT
name	VARCHAR(100)	NO		NULL	
email	VARCHAR(150)	NO	UNI	NULL	
role	INT	YES	MUL	NULL	
department_id	INT	YES	MUL	NULL	
created_at	DATETIME	YES		CURRENT_TIMESTAMP	
is_active	BOOLEAN	YES		TRUE	
FK: role → Roles(role_id) department_id → Departments(dept_id)					

```
mysql> DESCRIBE Purchase_Requests;
```

16 rows in table

Field	Type	Null	Key	Default	Extra
pr_id	INT	NO	PRI	NULL	AUTO_INCREMENT
employee_id	INT	YES	MUL	NULL	
dept_id	INT	YES	MUL	NULL	
item_name	VARCHAR(200)	NO		NULL	
description	TEXT	YES		NULL	
quantity	INT	NO		NULL	
estimated_cost	DECIMAL(12,2)	YES		NULL	
status	VARCHAR(20)	YES		'PENDING'	
created_at	DATETIME	YES		CURRENT_TIMESTAMP	

FK: employee_id → Users(user_id) | dept_id → Departments(dept_id)

```
mysql> DESCRIBE PR_Approvals;
```

10 rows in table

Field	Type	Null	Key	Default	Extra
approval_id	INT	NO	PRI	NULL	AUTO_INCREMENT
pr_id	INT	YES	MUL	NULL	
manager_id	INT	YES	MUL	NULL	
decision	VARCHAR(20)	YES		NULL	
comments	TEXT	YES		NULL	
decided_at	DATETIME	YES		CURRENT_TIMESTAMP	

FK: pr_id → Purchase_Requests(pr_id) | manager_id → Users(user_id)

```
mysql> DESCRIBE PR_Status_History;
```

1 row in table

Field	Type	Null	Key	Default	Extra
history_id	INT	NO	PRI	NULL	AUTO_INCREMENT
pr_id	INT	YES	MUL	NULL	
old_status	VARCHAR(20)	YES		NULL	
new_status	VARCHAR(20)	YES		NULL	
changed_by	INT	YES	MUL	NULL	
changed_at	DATETIME	YES		CURRENT_TIMESTAMP	

FK: pr_id → Purchase_Requests(pr_id) | changed_by → Users(user_id)

```
mysql> DESCRIBE Vendors;
```

4 rows in table

Field	Type	Null	Key	Default	Extra
vendor_id	INT	NO	PRI	NULL	AUTO_INCREMENT
company_name	VARCHAR(150)	NO		NULL	
contact_name	VARCHAR(100)	YES		NULL	
email	VARCHAR(150)	YES		NULL	
phone	VARCHAR(30)	YES		NULL	
address	VARCHAR(255)	YES		NULL	
rating	INT	YES		NULL	
is_active	BOOLEAN	YES		TRUE	

```
mysql> DESCRIBE Purchase_Orders;
```

5 rows in table

Field	Type	Null	Key	Default	Extra
po_id	INT	NO	PRI	NULL	AUTO_INCREMENT
pr_id	INT	YES	MUL	NULL	
vendor_id	INT	YES	MUL	NULL	
procurement_officer_id	INT	YES	MUL	NULL	
issue_date	DATETIME	YES		CURRENT_TIMESTAMP	
delivery_due_date	DATE	YES		NULL	
status	VARCHAR(20)	YES		'ISSUED'	
total_amount	DECIMAL(15,2)	YES		NULL	

FK: pr_id → Purchase_Requests(pr_id) | vendor_id → Vendors(vendor_id) | procurement_officer_id → Users(user_id)

```
mysql> DESCRIBE Invoices;
```

Field	Type	Null	Key	Default	Extra
invoice_id	INT	NO	PRI	NULL	AUTO_INCREMENT
po_id	INT	YES	MUL	NULL	
vendor_id	INT	YES	MUL	NULL	
invoice_date	DATETIME	YES		CURRENT_TIMESTAMP	
due_date	DATE	YES		NULL	
amount	DECIMAL(15,2)	YES		NULL	
status	VARCHAR(20)	YES		'PENDING'	

FK: po_id $\rightarrow$ Purchase_Orders(po_id) | vendor_id $\rightarrow$ Vendors(vendor_id)

```
mysql> DESCRIBE Goods_Receipt;
```

Field	Type	Null	Key	Default	Extra
gr_id	INT	NO	PRI	NULL	AUTO_INCREMENT
po_id	INT	YES	MUL	NULL	
received_by	INT	YES	MUL	NULL	
received_date	DATETIME	YES		CURRENT_TIMESTAMP	
quantity_received	INT	YES		NULL	
condition_notes	TEXT	YES		NULL	

FK: po_id → Purchase_Orders(po_id) | received_by → Users(user_id)

```
mysql> DESCRIBE Audit_Log;
```

Field	Type	Null	Key	Default	Extra
log_id	INT	NO	PRI	NULL	AUTO_INCREMENT
user_id	INT	YES	MUL	NULL	
action	VARCHAR(50)	YES		NULL	
table_affected	VARCHAR(50)	YES		NULL	
timestamp	DATETIME	YES		CURRENT_TIMESTAMP	
ip_address	VARCHAR(45)	YES		NULL	

FK: user_id $\rightarrow$ Users(user_id)

5. Query Pack (15 Queries) with Result Screenshots

This section presents 15 SQL queries written in MySQL syntax, organized into three tiers. Each query includes a comment header, the SQL code, and a **result output table** showing actual data returned from the populated database.

5.1 Basic Queries (Queries 1–8)

Query 1: List All Users With Their Roles and Departments

Retrieves user accounts joined with their assigned role name and department name.

BASIC — JOIN + ORDER BY

```
-- Query 1: List all users with their role name and department name
-- Uses: INNER JOIN (2 tables), ORDER BY
SELECT
    u.user_id,
    u.name      AS user_name,
    u.email,
    r.role_name,
    d.dept_name AS department
FROM Users u
    INNER JOIN Roles r      ON u.role = r.role_id
    INNER JOIN Departments d ON u.department_id = d.dept_id
ORDER BY u.name ASC;
```

► **Result: 6 rows returned**

user_id	user_name	email	role_name	department
6	Test Admin	admin@test.com	SuperAdmin	IT Department
1	Test Employee	employee@test.com	Employee	IT Department
5	Test Finance	finance@test.com	Finance	IT Department
2	Test Manager	manager@test.com	Manager	IT Department
3	Test Procure	procurement@test.com	Procurement	IT Department
4	Test Vendor	vendor@test.com	Vendor	IT Department

Query 2: All Pending Purchase Requests With Employee Details

Retrieves all purchase requests currently awaiting manager approval.

BASIC — JOIN + WHERE

```
-- Query 2: Pending purchase requests with employee name and department
-- Uses: INNER JOIN (2 tables), WHERE filter
SELECT
  pr.pr_id, u.name AS requested_by, d.dept_name,
  pr.item_name, pr.quantity, pr.estimated_cost, pr.created_at
FROM Purchase_Requests pr
  INNER JOIN Users u      ON pr.employee_id = u.user_id
  INNER JOIN Departments d ON pr.dept_id = d.dept_id
WHERE pr.status = 'PENDING'
ORDER BY pr.created_at DESC;
```

► Result: 6 rows returned

pr_id	requested_by	dept_name	item_name	qty	est_cost	created_at
1	Test Employee	IT Department	1	2	1200.00	2026-03-03 18:18:32
6	Test Employee	IT Department	Coffee Machine	3	300.00	2026-02-26 08:17:43
12	Test Employee	IT Department	Ergonomic Keyboard	3	80.00	2026-02-22 12:17:43
13	Test Employee	IT Department	Printer Ink	1	45.00	2026-02-18 20:17:43
15	Test Employee	IT Department	Coffee Machine	4	300.00	2026-02-12 15:17:43
8	Test Employee	IT Department	Coffee Machine	5	300.00	2026-02-07 09:17:43

Query 3: Purchase Orders With Vendor Company Name

Lists all purchase orders alongside the vendor and procurement officer. Sorted by issue date, most recent first.

BASIC — MULTI-JOIN + ORDER BY

```
-- Query 3: Purchase Orders with vendor and officer details
-- Uses: INNER JOIN (3 tables), ORDER BY
SELECT po.po_id, v.company_name AS vendor, u.name AS issued_by,
       po.total_amount, po.status, po.issue_date, po.delivery_due_date
FROM Purchase_Orders po
     INNER JOIN Vendors v ON po.vendor_id = v.vendor_id
     INNER JOIN Users u   ON po.procurement_officer_id = u.user_id
ORDER BY po.issue_date DESC;
```

► Result: 5 rows returned

po_id	vendor	issued_by	total_amount	status	issue_date	due_date
4	Enterprise Hardware	Test Procure	1000.00	SHIPPED	2026-02-21 15:17:43	2026-03-10
3	Tech Supplies Inc	Test Procure	160.00	PAID	2026-02-16 13:17:43	2026-03-10
1	Tech Supplies Inc	Test Procure	240.00	SHIPPED	2026-02-05 22:17:43	2026-03-10
2	Office Supplies Co	Test Procure	1500.00	ISSUED	2026-02-05 22:17:43	2026-03-10
5	Global Logistics	Test Procure	80.00	ISSUED	2026-02-02 22:17:43	2026-03-10

Query 4: Active Vendors Sorted by Rating

Lists all currently active vendors in descending order of star rating.

BASIC — FILTER + ORDER BY

```
-- Query 4: Active vendors sorted by rating (highest first)
-- Uses: WHERE filter, ORDER BY
SELECT vendor_id, company_name, contact_name, email, phone, rating
FROM Vendors
WHERE is_active = TRUE
ORDER BY rating DESC;
```

► Result: 4 rows returned

vendor_id	company_name	contact_name	email	phone	rating
1	Tech Supplies Inc	Alice	vendor@test.com	123-456	5
4	Enterprise Hardware	Dave	dave@eh.com	555-0103	5
2	Office Supplies Co	Bob	bob@office.com	555-0101	4
3	Global Logistics	Carol	carol@global.com	555-0102	3

Query 5: Approved Requests Awaiting PO Generation

Finds purchase requests approved but not yet converted into a purchase order.

BASIC — LEFT JOIN + WHERE NULL

```
-- Query 5: Approved requests without a purchase order yet
-- Uses: LEFT JOIN, WHERE ... IS NULL
SELECT pr.pr_id, pr.item_name, pr.quantity, pr.estimated_cost,
       u.name AS requested_by
FROM Purchase_Requests pr
     INNER JOIN Users u ON pr.employee_id = u.user_id
     LEFT JOIN Purchase_Orders po ON pr.pr_id = po.pr_id
WHERE pr.status = 'APPROVED' AND po.po_id IS NULL;
```

► Result: 0 rows returned (Empty set)

All approved requests have already been converted to purchase orders. This validates that the procurement workflow is operating correctly.

Query 6: Full Audit Log With User Email

Retrieves the 10 most recent audit log entries with user email, action, and IP address.

BASIC — JOIN + ORDER BY + LIMIT

```
-- Query 6: Recent audit log entries with user email
-- Uses: INNER JOIN, ORDER BY, LIMIT
SELECT a.log_id, u.email, a.action, a.table_affected,
       a.timestamp, a.ip_address
FROM Audit_Log a
     INNER JOIN Users u ON a.user_id = u.user_id
ORDER BY a.timestamp DESC LIMIT 10;
```

► Result: 9 rows returned

log_id	email	action	table_affected	timestamp	ip_address
3	employee@test.com	LOGOUT	Users	2026-03-03 18:19:20	127.0.0.1
2	employee@test.com	SUBMIT_PR	Purchase_Requests	2026-03-03 18:18:32	127.0.0.1
1	employee@test.com	LOGIN	Users	2026-03-03 18:18:12	127.0.0.1
9	admin@test.com	LOGIN	Users	2026-03-03 13:56:22	127.0.0.1
8	procurement@test.com	LOGOUT	Users	2026-03-03 13:44:54	127.0.0.1
7	procurement@test.com	LOGIN	Users	2026-03-03 13:44:43	127.0.0.1
6	manager@test.com	LOGOUT	Users	2026-03-03 13:44:37	127.0.0.1
5	manager@test.com	LOGIN	Users	2026-03-03 13:44:34	127.0.0.1
4	manager@test.com	LOGOUT	Users	2026-03-03 13:44:25	127.0.0.1

Query 7: Invoices With Purchase Order and Vendor Details

Joins invoices with their parent PO and billing vendor for accounts payable view.

BASIC — MULTI-JOIN

```
-- Query 7: Invoice details with PO reference and vendor
-- Uses: INNER JOIN (3 tables)
SELECT i.invoice_id, i.po_id, v.company_name AS vendor,
       i.amount, i.status AS invoice_status,
       po.status AS po_status, i.due_date
FROM Invoices i
     INNER JOIN Purchase_Orders po ON i.po_id = po.po_id
     INNER JOIN Vendors v         ON i.vendor_id = v.vendor_id
ORDER BY i.invoice_date DESC;
```

► Result: 1 row returned

invoice_id	po_id	vendor	amount	invoice_status	po_status	due_date
1	3	Tech Supplies Inc	160.00	PAID	PAID	2026-03-17

Query 8: Department Budget Overview

Calculates remaining budget and utilization percentage for every department.

BASIC — COMPUTED COLUMNS + ROUND()

```
-- Query 8: Department budget summary with remaining balance
-- Uses: Computed columns, arithmetic, ROUND()
SELECT dept_id, dept_name, budget_allocated, budget_used,
       (budget_allocated - budget_used) AS budget_remaining,
       ROUND((budget_used * 100.0 / budget_allocated), 2) AS utilization_pct
FROM Departments
WHERE budget_allocated > 0
ORDER BY utilization_pct DESC;
```

► Result: 3 rows returned

dept_id	dept_name	budget_allocated	budget_used	budget_remaining	utilization_pct
1	IT Department	100,000.00	160.00	99,840.00	0.16%
2	HR Department	50,000.00	0.00	50,000.00	0.00%
3	Finance Department	75,000.00	0.00	75,000.00	0.00%

5.2 Intermediate Queries (Queries 9–13)

Query 9: Purchase Request Count Per Status

Groups all purchase requests by status and counts each category.

INTERMEDIATE — GROUP BY + HAVING

```
-- Query 9: Count of purchase requests grouped by status
-- Uses: GROUP BY, HAVING, COUNT()
SELECT status, COUNT(*) AS request_count
FROM Purchase_Requests
GROUP BY status
HAVING COUNT(*) > 0
ORDER BY request_count DESC;
```

► Result: 3 rows returned

status	request_count
PENDING	6
REJECTED	5
APPROVED	5

Query 10: Total Purchase Order Value Per Vendor

Aggregates total PO value per vendor. Only includes vendors exceeding \$500.

INTERMEDIATE — GROUP BY + HAVING + SUM/AVG

```
-- Query 10: Total PO value per vendor (minimum $500)
-- Uses: INNER JOIN, GROUP BY, HAVING, SUM(), AVG()
SELECT v.company_name,
       COUNT(po.po_id)          AS total_orders,
       SUM(po.total_amount)     AS total_value,
       ROUND(AVG(po.total_amount), 2) AS avg_order_value
FROM Purchase_Orders po
     INNER JOIN Vendors v ON po.vendor_id = v.vendor_id
GROUP BY v.vendor_id, v.company_name
HAVING SUM(po.total_amount) > 500
ORDER BY total_value DESC;
```

► Result: 2 rows returned

company_name	total_orders	total_value	avg_order_value
Office Supplies Co	1	1,500.00	1,500.00
Enterprise Hardware	1	1,000.00	1,000.00

Query 11: Employees With More Than 3 Purchase Requests

Identifies high-volume requestors with their total request count and estimated spending.

INTERMEDIATE — GROUP BY + HAVING + COUNT/SUM

```
-- Query 11: High-volume requestors (more than 3 requests)
-- Uses: INNER JOIN, GROUP BY, HAVING, COUNT(), SUM()
SELECT u.name AS employee_name, u.email,
       COUNT(pr.pr_id) AS total_requests,
       SUM(pr.estimated_cost * pr.quantity) AS total_estimated_spend
FROM Purchase_Requests pr
     INNER JOIN Users u ON pr.employee_id = u.user_id
GROUP BY u.user_id, u.name, u.email
HAVING COUNT(pr.pr_id) > 3
ORDER BY total_requests DESC;
```

► Result: 1 row returned

employee_name	email	total_requests	total_estimated_spend
Test Employee	employee@test.com	16	26,190.00

Query 12: Vendors With Ratings Above the Average

Uses a subquery to find vendors rated above the system-wide average.

INTERMEDIATE — SUBQUERY

```
-- Query 12: Vendors rated above the system-wide average
-- Uses: Scalar subquery in WHERE clause
SELECT vendor_id, company_name, contact_name, rating
FROM Vendors
WHERE rating > (
    SELECT AVG(rating) FROM Vendors WHERE is_active = TRUE
)
ORDER BY rating DESC;
```

► Result: 2 rows returned

vendor_id	company_name	contact_name	rating
1	Tech Supplies Inc	Alice	5
4	Enterprise Hardware	Dave	5

Query 13: Departments Exceeding 50% Budget Utilization

Filters departments consuming more than half their allocated budget.

INTERMEDIATE — GROUP BY + HAVING (COMPUTED)

```
-- Query 13: Departments exceeding 50% budget utilization
-- Uses: GROUP BY, HAVING with arithmetic expression
SELECT dept_id, dept_name, budget_allocated, budget_used,
    ROUND((budget_used * 100.0 / budget_allocated), 2) AS utilization_pct
FROM Departments
WHERE budget_allocated > 0
GROUP BY dept_id, dept_name, budget_allocated, budget_used
HAVING (budget_used * 100.0 / budget_allocated) > 50.0
ORDER BY utilization_pct DESC;
```

► Result: 0 rows returned (Empty set)

No department has exceeded 50% budget utilization. The IT Department has only used 0.16% of its \$100,000 budget. This confirms healthy budget management across all departments.

5.3 Advanced / Bonus Queries (Queries 14–15)

Query 14: End-to-End Procurement Pipeline Report

Traces a purchase request from submission through approval, PO, goods receipt, and invoice — producing a comprehensive lifecycle view using a 6-table join chain.

ADVANCED — 6-TABLE JOIN

```
-- Query 14: Full procurement lifecycle report
-- Uses: 6-table INNER/LEFT JOIN chain across entire workflow
SELECT pr.pr_id, u_emp.name AS employee, pr.item_name,
       pr.quantity, pr.estimated_cost,
       pra.decision AS approval_decision,
       u_mgr.name AS approved_by, po.po_id,
       v.company_name AS vendor, po.total_amount AS po_value,
       po.status AS po_status, gr.received_date,
       gr.condition_notes, i.invoice_id,
       i.amount AS invoice_amount, i.status AS invoice_status
FROM Purchase_Requests pr
     INNER JOIN Users u_emp      ON pr.employee_id = u_emp.user_id
     LEFT JOIN PR_Approvals pra  ON pr.pr_id = pra.pr_id
     LEFT JOIN Users u_mgr      ON pra.manager_id = u_mgr.user_id
     LEFT JOIN Purchase_Orders po ON pr.pr_id = po.pr_id
     LEFT JOIN Vendors v        ON po.vendor_id = v.vendor_id
     LEFT JOIN Goods_Receipt gr  ON po.po_id = gr.po_id
     LEFT JOIN Invoices i       ON po.po_id = i.po_id
ORDER BY pr.pr_id LIMIT 10;
```

► Result: 10 rows returned (showing first 10 of lifecycle)

pr_id	employee	item_name	qty	decision	approved_by	po_id	vendor	po_value	po_status	inv_id	inv_status
1	Test Employee	1	2	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
2	Test Employee	Ergo Keyboard	1	REJECTED	Test Manager	NULL	NULL	NULL	NULL	NULL	NULL
5	Test Employee	Whiteboard	2	APPROVED	Test Manager	1	Tech Supplies	240.00	SHIPPED	NULL	NULL
7	Test Employee	Coffee Machine	5	APPROVED	Test Manager	2	Office Supplies	1500.00	ISSUED	NULL	NULL
9	Test Employee	Ergo Keyboard	2	APPROVED	Test Manager	3	Tech Supplies	160.00	PAID	1	PAID
10	Test Employee	Monitor 27"	4	APPROVED	Test Manager	4	Enterprise HW	1000.00	SHIPPED	NULL	NULL

Row pr_id=9 shows a completed lifecycle: APPROVED → PO_ISSUED → SHIPPED → PAID with invoice #1 settled.

Query 15: Vendors Who Have Never Received a Purchase Order

Uses a correlated NOT EXISTS subquery to find registered vendors with zero purchase orders.

ADVANCED — CORRELATED SUBQUERY (NOT EXISTS)

```
-- Query 15: Vendors with zero purchase orders (never used)
-- Uses: Correlated subquery with NOT EXISTS
SELECT v.vendor_id, v.company_name, v.contact_name,
       v.email, v.rating
FROM Vendors v
WHERE NOT EXISTS (
    SELECT 1 FROM Purchase_Orders po
    WHERE po.vendor_id = v.vendor_id
)
ORDER BY v.company_name;
```

► Result: 0 rows returned (Empty set)

All four registered vendors have received at least one purchase order, indicating healthy vendor utilization across the procurement pipeline.

Query Index Summary

#	Query Title	Category	SQL Features Used
1	Users With Roles & Departments	Basic	INNER JOIN (2), ORDER BY
2	Pending Purchase Requests	Basic	INNER JOIN (2), WHERE, ORDER BY
3	POs With Vendor Details	Basic	INNER JOIN (3), ORDER BY
4	Active Vendors by Rating	Basic	WHERE filter, ORDER BY
5	Approved Requests Awaiting PO	Basic	LEFT JOIN, WHERE IS NULL
6	Full Audit Log	Basic	INNER JOIN, ORDER BY, LIMIT
7	Invoices With PO and Vendor	Basic	INNER JOIN (3)
8	Department Budget Overview	Basic	Computed columns, ROUND()
9	Request Count Per Status	Intermediate	GROUP BY, HAVING, COUNT()
10	Total PO Value Per Vendor	Intermediate	GROUP BY, HAVING, SUM(), AVG()
11	High-Volume Requestors	Intermediate	GROUP BY, HAVING, COUNT(), SUM()
12	Vendors Above Average Rating	Intermediate	Scalar Subquery
13	Over-50% Budget Utilization	Intermediate	GROUP BY, HAVING (computed)
14	Full Procurement Pipeline	Advanced ★	6-table JOIN chain
15	Vendors With No Orders	Advanced ★	Correlated NOT EXISTS